

Título del trabajo:

Resumen del Curso de Introducción a Visual Basic: Variables, Operadores, Arrays, Bucles y Procedimientos

Nombre:

Juan Manuel Gómez Achucarro

Materia:

Lenguajes Visuales I

Curso:

2º Curso. Sección B. Turno Mañana

Profesor:

Marcos Contrera

Fecha:

26/04/2025

INTRODUCCIÓN

El presente trabajo tiene como objetivo resumir y analizar los contenidos presentados en los primeros diecinueve videos del curso de Introducción a Visual Basic. A lo largo de los mismos, se exploran los fundamentos esenciales para el desarrollo de programas en este lenguaje de programación.

Se comienza con una descripción general del curso y una introducción a las variables, continuando con los operadores aritméticos y lógicos, indispensables para el tratamiento de datos. Posteriormente, se abordan los arrays y los arrays multidimensionales, así como su utilización en programas básicos. También se presentan las estructuras condicionales, tanto simples como mediante el uso de la estructura Select Case, y los distintos tipos de bucles, haciendo especial énfasis en el bucle For.

En etapas más avanzadas, el curso introduce conceptos como el alcance de variables, el diseño de procedimientos y funciones, la utilización de parámetros opcionales y arrays como parámetros, y la sobrecarga de métodos. Finalmente, se ofrece una introducción al manejo de excepciones, brindando una base sólida para la gestión de errores en las aplicaciones.

El desarrollo de estos contenidos proporciona a los estudiantes los conocimientos esenciales para iniciarse en la programación estructurada utilizando Visual Basic, facilitando la comprensión de conceptos que serán fundamentales en niveles más avanzados.

Introducción al Curso (Video 1)

El curso de Visual Basic .NET se orienta a la formación de estudiantes en el desarrollo de aplicaciones de escritorio, partiendo desde un nivel inicial. Está dirigido tanto a personas sin conocimientos previos como a aquellas que poseen bases en programación.

Se empleará como entorno de desarrollo integrado (IDE) **Microsoft Visual Studio Community 2017**, una herramienta gratuita que ofrece soporte completo para Visual Basic .NET. Se detalla el procedimiento de descarga e instalación, haciendo hincapié en la posibilidad de personalizar los componentes instalados y en la necesidad de disponer de aproximadamente 3 GB de espacio libre.

El curso aborda temas fundamentales como:

- Estructura y sintaxis del lenguaje Visual Basic .NET.
- Conceptos de programación orientada a objetos: clases, objetos, encapsulamiento, herencia y polimorfismo.
- Creación de interfaces gráficas de usuario (GUI).
- Conexión de aplicaciones a bases de datos.
- Generación de instaladores para distribución de software.

Se destaca que Visual Basic .NET, al ser un lenguaje de alto nivel y basado en objetos, facilita tanto el aprendizaje de conceptos básicos como el abordaje de prácticas de programación modernas. La estructura del curso es progresiva, permitiendo una asimilación gradual de los contenidos.

Introducción a Visual Basic y Variables (Video 2)

En esta sección se introduce el concepto de **variable**, definida como un espacio en memoria destinado al almacenamiento de datos cuyo contenido puede cambiar durante la ejecución del programa. Asimismo, se distingue la **constante** como un espacio de memoria cuyo valor permanece inmutable.

Los tipos de datos se agrupan en tres categorías principales:

- **Numéricos:** Byte, Integer, Long, Decimal.
- **Texto:** String, Char.
- **Varios:** Boolean, Date (fecha y hora).

Se guía al estudiante en la creación de un nuevo proyecto en Visual Studio, seleccionando el tipo "Aplicación de Windows Forms" en Visual Basic, y se explican los principales componentes del entorno: el **cuadro de herramientas**, el **explorador de soluciones** y las **vistas de diseño y código**.

Se introduce la declaración de variables utilizando la palabra reservada Dim, estableciendo tanto el tipo de dato como un valor inicial, y se muestra cómo utilizar el método MsgBox para desplegar mensajes en pantalla. Asimismo, se subraya la importancia del uso de **comentarios** para documentar y clarificar el código.

La clase concluye con ejercicios prácticos de declaración y uso de variables de diferentes tipos, reforzando los conceptos introducidos.

Operadores Aritméticos (Video 3)

El tercer módulo presenta los **operadores aritméticos** disponibles en Visual Basic .NET, a saber:

- Suma (+)
- Resta (-)
- Multiplicación (*)
- División (/)
- Exponenciación (^)
- División entera (\)
- Módulo o resto de división (Mod)

Se desarrolla un proyecto denominado 'operadores', en el cual se ejemplifican las distintas operaciones utilizando variables de tipo numérico. Se analiza el impacto del tipo de dato en los resultados, especialmente en operaciones de división que involucren números decimales.

Se explican en detalle las diferencias entre:

- La **división estándar** (/), que conserva la parte decimal del resultado.
- La **división entera** (\), que descarta la parte decimal.
- El **operador Módulo** (Mod), que retorna el resto de la división, resultando útil para la implementación de algoritmos que requieran validaciones de paridad u otras condiciones.

Finalmente, se refuerzan buenas prácticas de programación tales como la actualización de variables en operaciones sucesivas y el uso estratégico de comentarios para depuración y documentación del código. La lección culmina con un repaso integral y la invitación a continuar con los contenidos posteriores del curso.

Video 4: Introducción a los Arrays

Introducción

- Se introduce el tema de arrays en el contexto de Visual Basic para aplicaciones de consola.
- Se aclara que se utilizará Console.WriteLine en lugar de MsgBox.

Configuración del Proyecto

- Creación de un nuevo proyecto de consola en Visual Studio 2017 llamado "video 3".

Creación de Variables

- Declaración de una variable tipo String utilizando Dim.
- Ejecución para mostrar el mensaje "Hola Mundo" en la consola.

Introducción a los Arrays

- Explicación de qué es un array y su diferencia respecto a una variable común.
- Sintaxis de declaración e inicialización de arrays.

Manipulación de Arrays

- Creación de un array llamado "mi array" con valores numéricos (10, 20, 30).
- Acceso a elementos mediante índices, recordando que el primer índice es 0.

Longitud y Redimensionamiento de Arrays

- Uso de .Length para obtener la cantidad de elementos.
- Uso de ReDim para cambiar el tamaño del array y ReDim Preserve para conservar los datos existentes.

Errores Comunes y Conclusiones

- Mención de errores típicos como el "índice fuera de los límites".
- Recomendación de práctica individual para reforzar los conceptos.

Video 5: Introducción a Arrays Multidimensionales

Introducción

- Bienvenida al tema de arrays multidimensionales.

Concepto de Arrays Unidimensionales

- Repaso de arrays simples y acceso por índices.

Creación de Arrays Bidimensionales

- Declaración de arrays bidimensionales especificando filas y columnas.

Arrays Multidimensionales y Rango

- Introducción al rango (cantidad de dimensiones) de los arrays.
- Mención de que se pueden crear hasta 32 dimensiones.

Práctica con Arrays en Código

- Creación de un array bidimensional para almacenar nombre, edad y teléfono de personas.

Asignación de Valores y Visualización

- Carga de datos en las posiciones correspondientes del array.
- Corrección de errores relacionados con índices fuera de rango.

Concatenación de Cadenas y Cierre

- Introducción a la concatenación de textos.
- Invitación a seguir practicando y continuar con el curso.

Video 6: Lectura de Datos y Estructuras Condicionales

Introducción al Curso de Visual Basic

- Creación de un proyecto de consola para realizar la suma de dos números ingresados por el usuario.

Lectura de Datos del Usuario

- Uso de `Console.ReadLine` para capturar entradas del usuario.

Almacenamiento de Datos en Variables

- Declaración e inicialización de variables para almacenar los números ingresados.

Realización de la Suma y Presentación del Resultado

- Uso de `Console.WriteLine` para mostrar la suma, inicialmente incorrecta por tratarse de texto.

Conversión de Tipos de Datos

- Uso de `Convert.ToInt32` para convertir los datos de texto a enteros y realizar correctamente la suma.

Conclusión y Avance en el Curso

- Resumen de los conceptos abordados y anuncio de futuros temas como interfaces gráficas.

Introducción a las Estructuras Condicionales

Operadores de Comparación

- Igual, distinto, menor, mayor, menor o igual, mayor o igual.
- Mención de otros tres operadores adicionales.

Creación de un Proyecto

- Creación de un proyecto de consola llamado "cajero automático".

Implementación del Menú Principal

- Presentación de un menú con opciones de consulta de saldo, ingreso, retiro y salida.

Almacenamiento de la Opción Elegida

- Captura de la opción seleccionada en una variable tipo `String`.

Uso de Condicionales: Bloque If

- Introducción al uso del bloque If ... End If para ejecutar acciones según condiciones.

Comparación de Opciones del Usuario

- Ejemplo práctico de comparación de la opción ingresada.

Condicionales Múltiples

- Implementación de múltiples condiciones con If...Elseif.

Finalización del Programa

- Comportamiento ante opciones no válidas y recomendaciones para validar entradas del usuario.

Video 7: Introducción a las Estructuras Condicionales – Select Case

Este video introduce el uso de estructuras condicionales en Visual Basic, enfocándose particularmente en la instrucción Select Case.

El presentador parte de un proyecto previo, en el cual se había utilizado un bloque If, para demostrar cómo la estructura Select Case permite organizar mejor múltiples condiciones relacionadas con el valor de una variable denominada opciónUsuario.

Cada caso dentro de Select Case responde a diferentes valores posibles de dicha variable, incluyendo un caso por defecto para manejar situaciones no previstas. Además, se explica la posibilidad de agrupar varias condiciones en un solo Case.

Posteriormente, se comienza a transformar el proyecto en un simulador de cajero automático. Para ello:

- Se crea una variable saldo de tipo decimal para representar cantidades monetarias, destacando la importancia de utilizar el tipo de dato adecuado.
- Se implementan operaciones básicas como ingreso y retiro de dinero, actualizando el saldo en función de las acciones del usuario.
- Se incorporan mejoras en la experiencia del usuario mediante mensajes de bienvenida, agradecimiento y despedida.

Finalmente, se señala que en futuros videos se continuará perfeccionando el programa hacia una versión más completa de un cajero automático.

Video 8: Introducción a los Operadores Lógicos

En este video se abordan los operadores lógicos fundamentales en la programación:

- **AND:** Devuelve verdadero únicamente si todas las condiciones son verdaderas.
- **OR:** Devuelve verdadero si al menos una condición es verdadera.

- **NOT:** Invierte el valor lógico de una condición.

Se presentan ejemplos prácticos, como el acceso a un curso condicionado a ser mayor de edad y/o poseer título de bachillerato.

Posteriormente, se implementan estos conceptos en el proyecto del cajero automático:

- Se valida que el número de tarjeta y el PIN introducidos por el usuario sean correctos utilizando operadores lógicos en estructuras If.
- Se demuestra el comportamiento del programa ante diversas combinaciones de entradas correctas e incorrectas.

El video concluye destacando la importancia de comprender y dominar el uso de operadores lógicos para la creación de aplicaciones robustas y funcionales.

Video 9: Introducción al Bucle For

Este video introduce el uso del bucle For, utilizado para ejecutar instrucciones repetidamente un número definido de veces.

Se explica su estructura básica:

- Inicialización de la variable de control.
- Condición de terminación.
- Incremento (o decremento) controlado, que puede modificarse mediante la cláusula Step.

Se muestra un ejemplo básico imprimiendo valores desde 0 hasta 10 en consola.

Posteriormente, se aplica el bucle For al proyecto del cajero automático:

- Se crea un array bidimensional que almacena múltiples tarjetas, cada una con su número, PIN y saldo asociados.
- Se recorre el array para validar el número de tarjeta y el PIN introducidos por el usuario.
- Si los datos coinciden, se permite el acceso y se muestra el saldo correspondiente.

Finalmente, se recomienda optimizar el código declarando variables únicamente cuando sean estrictamente necesarias, y se anticipa la introducción de nuevos bucles (Do While) en el próximo video.

Video 10: Configuración de Arrays y Lógica Principal del Programa

El décimo video del curso de Visual Basic .NET, desarrollado en Visual Studio 2017, tiene como objetivo completar la lógica principal del proyecto de un cajero automático, preparando el avance hacia funcionalidades futuras como "Book the Tube".

Inicialmente, se configura un array bidimensional de tres filas por tres columnas, donde cada fila representa una tarjeta con sus respectivos número, PIN y saldo. Posteriormente, se solicita al usuario el ingreso de un número de tarjeta y su PIN, los cuales se comparan

con los registros del array mediante un bucle for. Si los datos coinciden, se habilita el acceso al menú principal; de lo contrario, se notifica un error.

El menú permite consultar el saldo, ingresar dinero o realizar retiros. En el caso de ingreso, se solicita una cantidad sin conversión a decimal; en el caso de retiro, se valida que el monto no supere el saldo disponible. Finalmente, se indica que el menú deberá poder repetirse, funcionalidad que será abordada en el siguiente video.

Video 11: Introducción y Uso de Bucles

El undécimo video introduce el concepto de bucles en Visual Basic .NET, destacando su importancia en la repetición de operaciones hasta que el usuario decida finalizar el programa.

Se presenta inicialmente el bucle For, explicando su funcionamiento y la necesidad de establecer condiciones de salida para evitar bucles infinitos. Posteriormente, se explica el uso de instrucciones como While y Until, que permiten ejecutar bloques de código basados en condiciones lógicas.

A través de ejemplos prácticos, se implementa un bucle que permite validar repetidamente el número de tarjeta y PIN, y que mantiene activo el menú principal hasta que el usuario seleccione salir. Se destaca también la importancia de declarar y controlar correctamente las variables involucradas en el proceso.

Video 12: Alcance de Variables en Visual Basic

En el duodécimo video, se inicia un nuevo proyecto en Visual Studio, con el propósito de abordar el concepto de alcance de las variables.

Se explica que en Visual Basic las instrucciones se agrupan en bloques de código, y que las variables declaradas dentro de un bloque solo existen mientras la ejecución permanezca en dicho bloque. Se emplean ejemplos prácticos para demostrar que una variable pierde su existencia fuera del bloque donde fue creada, lo que puede generar errores si se intenta acceder a ella.

Finalmente, se enfatiza la importancia de comprender el alcance de las variables como base fundamental antes de introducirse en temas más avanzados, tales como procedimientos y funciones. El video concluye incentivando a los estudiantes a seguir profundizando en los contenidos del curso.

Video 13: Introducción a los Procedimientos

Presentación del Tema

El video se centra en la enseñanza de los procedimientos en programación, utilizando Visual Basic en el entorno de desarrollo Visual Studio 2017. Para ilustrar los conceptos, se crea un proyecto de aplicación de consola.

Concepto de Procedimiento

Se define un procedimiento o subrutina como un bloque de instrucciones que se ejecuta únicamente cuando es invocado desde el programa principal (método Main). En caso de no ser llamado, el procedimiento no se ejecuta.

Ejemplo de Procedimiento

Se ejemplifica el uso de procedimientos mediante la creación de un encabezado que se imprime en la consola utilizando Console.WriteLine. Para evitar la repetición de instrucciones, se encapsula este código dentro de un procedimiento denominado Encabezado, el cual puede ser invocado cuantas veces sea necesario.

Llamadas a Procedimientos

Se explica el mecanismo de llamada a procedimientos desde el método principal, demostrando cómo su ejecución puede ser repetida mediante múltiples invocaciones, promoviendo así la reutilización del código y reduciendo la redundancia.

Uso de Parámetros

Se introduce el concepto de parámetros, los cuales permiten enviar información a un procedimiento. Se desarrolla un ejemplo en el que se define un parámetro de tipo texto (texto) para personalizar el contenido del encabezado.

Procedimiento de Multiplicación

Se presenta un procedimiento denominado Multiplica, que solicita al usuario dos valores numéricos y muestra el resultado de su multiplicación, ejemplificando la versatilidad y la utilidad de los procedimientos para tareas dinámicas y repetitivas.

Conclusiones

El video finaliza resaltando la importancia de los procedimientos para la organización, eficiencia y reutilización del código. Se anticipa que en el siguiente video se abordará el tema de las funciones.

Video 14: Introducción a los Procedimientos (Continuación)

Introducción

Se da la bienvenida a los participantes, retomando el curso de Visual Basic .NET, y se informa que se trabajará con un proyecto de aplicación de consola en Visual Studio 2017.

Definición y Utilidad de Procedimientos

Se reafirma que un procedimiento es un bloque de instrucciones que se ejecuta únicamente cuando se le invoca, facilitando la organización del código y evitando su repetición.

Creación y Llamada de Procedimientos

Se muestra cómo crear el procedimiento Encabezado e invocarlo desde el método Main, optimizando así la estructura del programa mediante la reutilización del mismo bloque de instrucciones.

Parámetros en Procedimientos

Se introduce la posibilidad de incluir parámetros en los procedimientos, permitiendo que estos se adapten a diferentes datos en cada llamada. Se ejemplifica con un procedimiento que recibe un parámetro de tipo texto para personalizar el encabezado mostrado en consola.

Procedimiento de Multiplicación

Se presenta nuevamente el procedimiento Multiplica, que solicita dos números al usuario y calcula su producto, destacando la aplicación de procedimientos en operaciones matemáticas básicas.

Conclusiones

El video concluye subrayando la importancia de los procedimientos en la simplificación y mejor organización del código. Se anuncia que en el próximo video se abordará el estudio de las funciones, las cuales presentan diferencias importantes respecto a los procedimientos.

Video 15: Introducción a las Funciones

Introducción y Contextualización

El video inicia con una disculpa por el periodo de inactividad en la publicación de contenidos, manifestando la intención de retomar el curso. Se utiliza Visual Studio 2019 para el desarrollo de los ejemplos, recomendando la actualización del software.

Configuración del Proyecto

Se crea un nuevo proyecto de tipo aplicación de consola en Visual Basic dentro del .NET Framework, nombrado 'funciones'.

Ejemplo de Cálculo de Área de un Triángulo

Se desarrolla un programa que calcula el área de un triángulo. Para ello, se solicita al usuario ingresar los valores de alto y ancho, que se convierten a tipo entero mediante `Convert.ToInt32`.

Procedimiento para el Cálculo del Área

Se crea un procedimiento que recibe el alto y el ancho como parámetros, calcula el área utilizando la fórmula $(\text{alto} * \text{ancho}) / 2$, y muestra el resultado en consola.

Introducción a las Funciones

Se explica que, a diferencia de los procedimientos, las funciones devuelven un valor tras su ejecución. Se ilustra cómo se puede definir una función que recibe parámetros, realiza un cálculo y retorna un resultado, permitiendo almacenar dicho valor en una variable.

Implementación de la Función `áreaTriángulo`

Se define una función denominada `áreaTriángulo`, que calcula y retorna el área del triángulo. Se utiliza la palabra clave `Return` para devolver el valor, y se almacena el resultado en la variable `miÁrea`.

Presentación de Resultados

Se enseña cómo imprimir el resultado en la consola utilizando concatenación de cadenas e interpolación, mejorando así la legibilidad y eficiencia del código.

Conclusión

El video finaliza con una recapitulación de los temas tratados, enfatizando la diferencia entre procedimientos y funciones. Se agradece a los espectadores su atención y se los invita a continuar con el aprendizaje en los próximos videos.

Video 16 — Introducción al Curso y Creación de Funciones

Introducción

El video forma parte de un curso introductorio de Visual Basic .NET orientado a principiantes, centrado en el paso de parámetros en funciones. El presentador da la bienvenida a los espectadores e indica que se trata del video número 15 del curso.

Creación de un Nuevo Proyecto

Se inicia la apertura de Visual Studio, donde se crea un nuevo proyecto de tipo *Aplicación de Consola* en Visual Basic, denominado “parámetros”.

Ejemplo de Función

Se propone un ejercicio sencillo: solicitar al usuario que introduzca un número, el cual será utilizado como parámetro en una función. Esta función multiplicará el número por dos y devolverá el resultado.

Desarrollo del Programa

Se explica el procedimiento para solicitar un número al usuario a través de la consola, almacenarlo en una variable denominada número, y convertir el valor introducido en un tipo de dato entero.

Declaración y Uso de la Función

Se declara una función denominada doble, que recibe un parámetro de tipo entero y devuelve un entero. Posteriormente, se crea una variable resultado que almacena el valor devuelto por la función, para luego ser mostrado en la consola junto con un mensaje descriptivo.

Concepto de Paso de Parámetros por Valor

Se introduce la noción de paso de parámetros por valor, destacando que los cambios en el parámetro no afectan el valor original de la variable.

Visualización de la Memoria

Se realiza una explicación conceptual sobre la asignación de memoria a variables y parámetros, describiendo cómo el paso de un parámetro implica la creación de una copia en memoria.

Errores Lógicos Comunes

Se discute un posible error lógico relacionado con el cambio de paso de parámetros de *por valor* a *por referencia*, ilustrando las consecuencias de una implementación incorrecta.

Conclusión

El video finaliza con un resumen de lo aprendido, y anticipa que en el siguiente encuentro se abordarán los parámetros opcionales y los parámetros *ByRef*.

Video 17 — Parámetros Opcionales y Arrays como Parámetros

Introducción

El video continúa el curso de Visual Basic .NET, marcando el avance al video número 16, y retoma conceptos relacionados con el paso de parámetros.

Nuevo Proyecto

Se crea un nuevo proyecto en Visual Studio denominado "parámetros 2", también de tipo *Aplicación de Consola*.

Parámetros Opcionales

Se introduce la utilización de parámetros opcionales mediante la creación de una función suma, la cual recibe tres parámetros: dos obligatorios (nombre y tiempo) y uno opcional (valor), con valor predeterminado de 2. Se aclara que los parámetros opcionales deben declararse al final de la lista de parámetros.

Desarrollo de la Función

Dentro de la función, se declara una variable `miSuma` que representa la suma de los parámetros. Se demuestra cómo llamar a la función utilizando únicamente los parámetros obligatorios, haciendo uso automático del valor por defecto.

Pruebas y Resultados

Se ejecutan pruebas para observar el comportamiento de la función con y sin la especificación del tercer parámetro, verificando el uso correcto del valor predeterminado.

Parámetros de Tipo Array

Se presenta el concepto de pasar un array como parámetro, utilizado cuando no se conoce previamente la cantidad de argumentos. Se desarrolla una función denominada promedio que calcula el promedio de los valores recibidos en un array.

Implementación de la Función Promedio

Se detalla la estructura de la función: inicialización de variables, uso de un bucle para recorrer el array, suma de los elementos y cálculo del promedio.

Diferencias entre Array y ParamArray

Se clarifica la diferencia entre declarar un parámetro como array, que exige una instancia previa, y el uso de `ParamArray`, que permite pasar múltiples argumentos sin crear un array explícito.

Conclusión

El video concluye resumiendo los conceptos abordados y agradeciendo a los participantes, invitándolos a continuar el aprendizaje en las siguientes entregas.

Video 18 — Sobrecarga de Métodos en Visual Basic

Introducción

El video da la bienvenida a los espectadores e introduce el tema de la **sobrecarga de métodos** dentro del curso de Visual Basic .NET, en el contexto del video número 17.

Creación del Proyecto

Se crea un nuevo proyecto de tipo *Aplicación de Consola* en Visual Studio, donde se ejemplificará el cálculo de promedios.

Definición de la Función Promedio

Se define una función denominada promedio, que recibe dos parámetros de tipo entero y devuelve un valor de tipo doble, permitiendo resultados decimales.

Implementación en el Método Main

En el método principal (Main), se declaran variables para almacenar y mostrar el resultado, se invoca la función promedio, y se imprime el resultado en consola.

Introducción a la Sobrecarga de Métodos

Se presenta la necesidad de calcular el promedio de tres números, para lo cual se crea una segunda función promedio, esta vez con tres parámetros.

Se destaca cómo Visual Basic selecciona automáticamente la función correcta basada en la cantidad de argumentos proporcionados.

Manejo de Errores y Buenas Prácticas

Se advierte sobre posibles errores de ambigüedad cuando se crean funciones sobrecargadas con igual número y tipo de parámetros, y se resalta la importancia de definir correctamente las firmas de las funciones.

Conclusión

Se finaliza enfatizando que comprender la sobrecarga de métodos es esencial para profundizar en la programación orientada a objetos.

Video 19 — Introducción al Manejo de Excepciones

Introducción

El video introduce el concepto de **excepciones** en Visual Basic .NET, describiéndolas como errores que se producen durante la ejecución de un programa, a pesar de que la sintaxis sea correcta.

Creación del Proyecto

Se crea un nuevo proyecto de tipo *Aplicación de Consola* para implementar un ejemplo práctico que abordará el manejo de errores mediante excepciones.

Ejemplo de Manejo de Excepciones

Se desarrolla una función llamada dividir que toma dos parámetros, realiza una división, y devuelve el resultado. Se destaca que errores como introducir un valor no numérico pueden generar excepciones.

Uso de Try-Catch

Se introduce la estructura de control *try-catch*, la cual permite capturar excepciones durante la ejecución:

- El bloque try contiene el código susceptible de producir errores.
- El bloque catch captura y maneja las excepciones que puedan surgir.

Consideraciones en la Implementación

Se enfatiza la necesidad de declarar correctamente las variables para evitar errores de ámbito, y se ejemplifica la correcta utilización del bloque try-catch.

CONCLUSIÓN

El estudio de los contenidos presentados en los diecinueve videos ha permitido adquirir una visión integral sobre la programación básica en Visual Basic. A través del abordaje progresivo de temas, desde variables y operadores hasta estructuras de control, arrays, procedimientos y manejo de excepciones, se establecieron los pilares necesarios para desarrollar programas funcionales y organizados.

La comprensión de la sintaxis y de la lógica de programación en Visual Basic resulta esencial no solo para el uso de este lenguaje, sino también como base para el aprendizaje de otros lenguajes de programación. Asimismo, la correcta utilización de funciones, procedimientos, bucles y estructuras condicionales fortalece el pensamiento lógico y la capacidad de resolución de problemas.

En conclusión, este curso introductorio representa un primer paso fundamental en el camino hacia un dominio más avanzado de la programación, proporcionando herramientas teóricas y prácticas que serán de gran utilidad en el desarrollo académico y profesional.